

LAPORAN PRAKTIKUM – BTREE

Laporan ini disusun untuk memenuhi Tugas Ke Sepuluh Praktikum Algoritma &

Pemrograman

Dosen Pengampu : Dr. Ionia Veritawati, S.Si., MT.



Oleh :

Nama : Khairul Adam Efendi

NPM : 4525210035

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS PANCASILA

2025/2026

Soal BTree_08 (nama file: PASD10-08.cpp):

- **Source Code**

8. Berdasarkan contoh no.2 pengurutan BTREE, buat dengan tanpa inputan (nama file : PASD10-08.cpp):

```
#include<iostream>

using namespace std;

struct CanBTree{
    int *d;
    CanBTree **CanAnakPointer;
    bool l;
    int n;
}*r=NULL,*np=NULL,*x=NULL;

CanBTree* init(){
    int i;
    np=new CanBTree;
    np->d=new int[6];
    np->CanAnakPointer=new CanBTree*[7];
    np->l=true;
    np->n=0;
    for(i=0;i<7;i++)
        np->CanAnakPointer[i]=NULL;
    return np;
}

void PohonB(CanBTree *p){
    cout<<endl;
    int i;
    for(i=0;i<p->n;i++){
        if(p->l==false)
            PohonB(p->CanAnakPointer[i]);
```

```

        cout<<" "<<p->d[i];
    }
    if(p->l==false)
        PohonB(p->CanAnakPointer[i]);
    cout<<endl;
}

void Urut(int *p,int n){
    int i,j,t;
    for(i=0;i<n;i++){
        for(j=i;j<=n;j++){
            if(p[i]>p[j]){
                t=p[i];
                p[i]=p[j];
                p[j]=t;
            }
        }
    }
}

int PecahAnak(CanBTree *x,int i){
    int j,mid;
    CanBTree *np1,*np3,*y;
    np3=init();
    np3->l=true;
    if(i==-1){
        mid=x->d[2];
        x->d[2]=0;
        x->n--;
        for(j=3;j<6;j++){
            np3->d[j-3]=x->d[j];
            np3->CanAnakPointer[j-3]=x->CanAnakPointer[j];
            np3->n++;
        }
    }
}

```

```

        x->d[j]=0;
        x->n--;
    }
    for(j=0;j<6;j++)
        x->CanAnakPointer[j]=NULL;
    np1=init();
    np1->d[0]=mid;
    np1->CanAnakPointer[np1->n]=x;
    np1->CanAnakPointer[np1->n+1]=np3;
    np1->n++;
    r=np1;
} else{
    y=x->CanAnakPointer[i];
    mid=y->d[2];
    y->d[2]=0;
    y->n--;
    for(j=3;j<6;j++){
        np3->d[j-3]=y->d[j];
        np3->n++;
        y->d[j]=0;
        y->n--;
    }
    x->CanAnakPointer[i+1]=y;
    x->CanAnakPointer[i+1]=np3;
}
return mid;
}

```

```

void Sisip(int a){
    int i,t;
    x=r;
    if(x==NULL){
        r=init();
    }
}

```

```

    x=r;
}
else{
    if(x->l==true && x->n==6){
        t=PecahAnak(x,-1);
        x=r;
        for(i=0;i<(x->n);i++){
            if((a>x->d[i]) && (a<x->d[i+1])){
                i++;
                break;
            }
            else if(a<x->d[0])
                break;
        }
        x=x->CanAnakPointer[i];
    }
    else{
        while(x->l==false){
            for(i=0;i<(x->n);i++){
                if((a>x->d[i]) && (a<x->d[i+1])){
                    i++;
                    break;
                }
                else if(a<x->d[0])
                    break;
            }
            if((x->CanAnakPointer[i])->n==6){
                t=PecahAnak(x,i);
                x->d[x->n]=t;z
                x->n++;
                continue;
            }
            else

```

```

        x=x->CanAnakPointer[i];

    }

}

x->d[x->n]=a;
Urut(x->d,x->n);
x->n++;
}

int main(){
    int data[]={2,3,1,4,5};
    int n=sizeof(data)/sizeof(data[0]);
    for(int i=0;i<n;i++)
        Sisip(data[i]);
    cout<<endl;
    cout<<"HASIL PENGURUTAN MENGGUNAKAN BTREE"<<endl;
    cout<<"~~~~~"<<endl;
    PohonB(r);
    return 0;
}

```

- SS Program

```
#include<iostream>
using namespace std;

struct CanBTree{
    int *d;
    CanBTree **CanAnakPointer;
    bool l;
    int n;
}*r=NULL,*np=NULL,*x=NULL;

CanBTree* init(){
    int i;
    np=new CanBTree;
    np->d=new int[6];
    np->CanAnakPointer=new CanBTree*[7];
    np->l=true;
    np->n=0;
    for(i=0;i<7;i++)
        np->CanAnakPointer[i]=NULL;
    return np;
}

void PohonB(CanBTree *p){
    cout<<endl;
    int i;
    for(i=0;i<p->n;i++){
        if(p->l==false)
            PohonB(p->CanAnakPointer[i]);
        cout<<" "<<p->d[i];
    }
    if(p->l==false)
        PohonB(p->CanAnakPointer[i]);
    cout<<endl;
}

void Urut(int *p,int n){
    int i,j,t;
    for(i=0;i<n;i++){
        for(j=i;j<=n;j++){
            if(p[i]>p[j]){
                t=p[i];
                p[i]=p[j];
                p[j]=t;
            }
        }
    }
}
```

```

}

int PecahAnak(CanBTree *x,int i){
    int j,mid;
    CanBTree *np1,*np3,*y;
    np3=init();
    np3->l=true;
    if(i==1){
        mid=x->d[2];
        x->d[2]=0;
        x->n--;
        for(j=3;j<6;j++){
            np3->d[j-3]=x->d[j];
            np3->CanAnakPointer[j-3]=x->CanAnakPointer[j];
            np3->n++;
            x->d[j]=0;
            x->n--;
        }
        for(j=0;j<6;j++){
            x->CanAnakPointer[j]=NULL;
        }
        np1=init();
        np1->d[0]=mid;
        np1->CanAnakPointer[np1->n]=x;
        np1->CanAnakPointer[np1->n+1]=np3;
        np1->n++;
        r=np1;
    }else{
        y=x->CanAnakPointer[i];
        mid=y->d[2];
        y->d[2]=0;
        y->n--;
        for(j=3;j<6;j++){
            np3->d[j-3]=y->d[j];
            np3->n++;
            y->d[j]=0;
            y->n--;
        }
        x->CanAnakPointer[i+1]=y;
        x->CanAnakPointer[i+1]=np3;
    }
    return mid;
}

void Sisip(int a){

```



```

int i,t;
x=r;
if(x==NULL){
    r=init();
    x=r;
}
else{
    if(x->l==true && x->n==6){
        t=PecahAnak(x,-1);
        x=r;
        for(i=0;i<(x->n);i++){
            if((a>x->d[i]) && (a<x->d[i+1])){
                i++;
                break;
            }
            else if(a<x->d[0])
                break;
        }
        x=x->CanAnakPointer[i];
    }
    else{
        while(x->l==false){
            for(i=0;i<(x->n);i++){
                if((a>x->d[i]) && (a<x->d[i+1])){
                    i++;
                    break;
                }
                else if(a<x->d[0])
                    break;
            }
            if((x->CanAnakPointer[i])->n==6){
                t=PecahAnak(x,i);
                x->d[x->n]=t;
                x->n++;
                continue;
            }
            else
                x=x->CanAnakPointer[i];
        }
    }
    x->d[x->n]=a;
    Urut(x->d,x->n);
    x->n++;
}

```

```

-}

int main() {
    int data[]={2,3,1,4,5};
    int n=sizeof(data)/sizeof(data[0]);
    for(int i=0;i<n;i++)
        Sisip(data[i]);
    cout<<endl;
    cout<<"HASIL PENGURUTAN MENGGUNAKAN BTREE"<<endl;
    cout<<"~~~~~"<<endl;
    PohonB(r);
    return 0;
-}

```

- **SS Running Program**

```

D:\ASD SMT2\ASD-A-25-26\PRAK10_BTree_Asprak01_4525210035_Khairul Adam Efendi>PASD10-08
HASIL PENGURUTAN MENGGUNAKAN BTREE
~~~~~
1 2 3 4 5

```

Soal BTree_11 (nama file: PASD10-11.cpp):

- **Source Code**

11. Perbaiki kesalahan coding dibawah ini (nama file : PASD10-11.ccp):

```
#include<iostream>
#include<cstdlib>
using namespace std;

struct Node{
    struct Node *Left;
    char INFO;
    struct Node *Right;
};

typedef struct Node Simpul;

Simpul *Root,*P,*Current;
Simpul *Q[129];

void Inisialisasi(){
    Root=NULL;
    P=NULL;
    Current=NULL;
    for(int i=0;i<129;i++){
        Q[i]=NULL;
    }
}

void BuatSimpul(char X){
    P=new Simpul;
    if(P!=NULL){
        P->INFO=X;
        P->Left=NULL;
        P->Right=NULL;
```

```

    }
    else{
        cout<<"Memory Heap Full"<<endl;
        exit(1);
    }
}

void BuatSimpulAkar(){
    if(Root==NULL){
        if(P!=NULL){
            Root=P;
            Root->Left=NULL;
            Root->Right=NULL;
            cout<<"Root Berhasil Dibuat"<<endl;
        }
        else{
            cout<<"Simpul Belum Dibuat"<<endl;
        }
    }
    else{
        cout<<"Root Sudah Ada"<<endl;
    }
}

bool IsEmpty(){
    if(Root==NULL)
        return true;
    else
        return false;
}

Simpul *Alokasi(char X){
    Simpul *NodeBaru;

```

```

NodeBaru=new Simpul;
if(NodeBaru!=NULL){
    NodeBaru->INFO=X;
    NodeBaru->Left=NULL;
    NodeBaru->Right=NULL;
}
return NodeBaru;
}

void InsertUrutNomor(){
    int depan=1;
    int belakang=1;
    char X;

    Q[belakang]=Root;

    while(depan<=belakang){
        Current=Q[depan];

        cout<<"Masukkan Anak Kiri dari "<<Current->INFO<<" (0 jika
tidak ada) : ";
        cin>>X;
        if(X!='0'){
            Current->Left=Alokasi(X);
            belakang++;
            Q[belakang]=Current->Left;
        }

        cout<<"Masukkan Anak Kanan dari "<<Current->INFO<<" (0
jika tidak ada) : ";
        cin>>X;
        if(X!='0'){
            Current->Right=Alokasi(X);

```

```

        belakang++;
        Q[belakang]=Current->Right;
    }

    depan++;
}
}

void BacaUrutNomor(){
    int depan=1;
    int belakang=1;

    Q[belakang]=Root;

    cout<<endl;
    cout<<"===== "<<endl;
    cout<<"ISI BINARY TREE (LEVEL ORDER)"<<endl;
    cout<<"===== "<<endl;

    while(depan<=belakang){
        Current=Q[depan];

        if(Current!=NULL){
            cout<<Current->INFO<<" ";

            if(Current->Left!=NULL){
                belakang++;
                Q[belakang]=Current->Left;
            }

            if(Current->Right!=NULL){
                belakang++;
                Q[belakang]=Current->Right;
            }
        }
    }
}

```

```

        }

    }

    depan++;

}

cout<<endl;
}

void PreOrder(Simpul *Root){
    if(Root!=NULL){
        cout<<Root->INFO<<" ";
        PreOrder(Root->Left);
        PreOrder(Root->Right);
    }
}

void InOrder(Simpul *Root){
    if(Root!=NULL){
        InOrder(Root->Left);
        cout<<Root->INFO<<" ";
        InOrder(Root->Right);
    }
}

void PostOrder(Simpul *Root){
    if(Root!=NULL){
        PostOrder(Root->Left);
        PostOrder(Root->Right);
        cout<<Root->INFO<<" ";
    }
}

```

```

int HitungNode(Simpul *Root){
    if(Root==NULL)
        return 0;
    return 1+HitungNode(Root->Left)+HitungNode(Root->Right);
}

int HitungDaun(Simpul *Root){
    if(Root==NULL)
        return 0;
    if(Root->Left==NULL && Root->Right==NULL)
        return 1;
    return HitungDaun(Root->Left)+HitungDaun(Root->Right);
}

int TinggiTree(Simpul *Root){
    if(Root==NULL)
        return 0;

    int kiri=TinggiTree(Root->Left);
    int kanan=TinggiTree(Root->Right);

    if(kiri>kanan)
        return kiri+1;
    else
        return kanan+1;
}

int main(){
    char X;

    Inisialisasi();

    cout<<"===== "<<endl;

```



```

cout<<"    PROGRAM BINARY TREE"<<endl;
cout<<"===== "<<endl;
cout<<"Masukkan Root : ";
cin>>X;

    BuatSimpul(X);
    BuatSimpulAkar();

cout<<endl;
InsertUrutNomor();

cout<<endl;
BacaUrutNomor();

cout<<endl;
cout<<"===== "<<endl;
cout<<"PREORDER"<<endl;
cout<<"===== "<<endl;
PreOrder(Root);

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INORDER"<<endl;
cout<<"===== "<<endl;
InOrder(Root);

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"POSTORDER"<<endl;

```

```
cout<<"===== "<<endl;
PostOrder(Root);

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INFORMASI TREE"<<endl;
cout<<"===== "<<endl;
cout<<"Jumlah Node : "<<HitungNode(Root)<<endl;
cout<<"Jumlah Daun : "<<HitungDaun(Root)<<endl;
cout<<"Tinggi Tree : "<<TinggiTree(Root)<<endl;

cout<<endl;
system("pause");
return 0;
}
```

- SS Program

```
#include<iostream>
#include<cstdlib>
using namespace std;

struct Node{
    struct Node *Left;
    char INFO;
    struct Node *Right;
};

typedef struct Node Simpul;

Simpul *Root,*P,*Current;
Simpul *Q[129];

void Inisialisasi(){
    Root=NULL;
    P=NULL;
    Current=NULL;
    for(int i=0;i<129;i++){
        Q[i]=NULL;
    }
}

void BuatSimpul(char X){
    P=new Simpul;
    if(P!=NULL){
        P->INFO=X;
        P->Left=NULL;
        P->Right=NULL;
    }
    else{
        cout<<"Memory Heap Full"<<endl;
        exit(1);
    }
}

void BuatSimpulAkar(){
    if(Root==NULL){
        if(P!=NULL){
            Root=P;
            Root->Left=NULL;
            Root->Right=NULL;
            cout<<"Root Berhasil Dibuat"<<endl;
        }
        else{
            cout<<"Simpul Belum Dibuat"<<endl;
        }
    }
    else{
```

```

        cout<<"Root Sudah Ada"<<endl;
    }
}

bool IsEmpty() {
    if (Root==NULL)
        return true;
    else
        return false;
}

Simpul *Alokasi(char X) {
    Simpul *NodeBaru;
    NodeBaru=new Simpul;
    if (NodeBaru!=NULL) {
        NodeBaru->INFO=X;
        NodeBaru->Left=NULL;
        NodeBaru->Right=NULL;
    }
    return NodeBaru;
}

void InsertUrutNomor() {
    int depan=1;
    int belakang=1;
    char X;

    Q[belakang]=Root;

    while (depan<=belakang) {
        Current=Q[depan];

        cout<<"Masukkan Anak Kiri dari "<<Current->INFO<<" (0 jika tidak ada) : ";
        cin>>X;
        if (X!='0') {
            Current->Left=Alokasi(X);
            belakang++;
            Q[belakang]=Current->Left;
        }

        cout<<"Masukkan Anak Kanan dari "<<Current->INFO<<" (0 jika tidak ada) : ";
        cin>>X;
        if (X!='0') {
            Current->Right=Alokasi(X);
            belakang++;
            Q[belakang]=Current->Right;
        }

        depan++;
    }
}

```

```

}

void BacaUrutNomor() {
    int depan=1;
    int belakang=1;

    Q[belakang]=Root;

    cout<<endl;
    cout<<"===== "<<endl;
    cout<<"ISI BINARY TREE (LEVEL ORDER) "<<endl;
    cout<<"===== "<<endl;

    while (depan<=belakang) {
        Current=Q[depan];

        if (Current!=NULL) {
            cout<<Current->INFO<<" ";

            if (Current->Left!=NULL) {
                belakang++;
                Q[belakang]=Current->Left;
            }

            if (Current->Right!=NULL) {
                belakang++;
                Q[belakang]=Current->Right;
            }
        }

        depan++;
    }

    cout<<endl;
}

void PreOrder(Simpul *Root) {
    if (Root!=NULL) {
        cout<<Root->INFO<<" ";
        PreOrder (Root->Left);
        PreOrder (Root->Right);
    }
}

void InOrder(Simpul *Root) {
    if (Root!=NULL) {
        InOrder (Root->Left);
        cout<<Root->INFO<<" ";
        InOrder (Root->Right);
    }
}

```

```

}

void PostOrder(Simpul *Root){
    if(Root!=NULL){
        PostOrder(Root->Left);
        PostOrder(Root->Right);
        cout<<Root->INFO<<" ";
    }
}

int HitungNode(Simpul *Root){
    if(Root==NULL)
        return 0;
    return 1+HitungNode(Root->Left)+HitungNode(Root->Right);
}

int HitungDaun(Simpul *Root){
    if(Root==NULL)
        return 0;
    if(Root->Left==NULL && Root->Right==NULL)
        return 1;
    return HitungDaun(Root->Left)+HitungDaun(Root->Right);
}

int TinggiTree(Simpul *Root){
    if(Root==NULL)
        return 0;

    int kiri=TinggiTree(Root->Left);
    int kanan=TinggiTree(Root->Right);

    if(kiri>kanan)
        return kiri+1;
    else
        return kanan+1;
}

int main(){
    char X;

    Inisialisasi();

    cout<<"===== "<<endl;
    cout<<"      PROGRAM BINARY TREE"<<endl;
    cout<<"===== "<<endl;
    cout<<"Masukkan Root : ";
    cin>>X;

    BuatSimpul(X);
    BuatSimpulAkar();
}

```

```

cout<<endl;
InsertUrutNomor ();

cout<<endl;
BacaUrutNomor ();

cout<<endl;
cout<<"===== "<<endl;
cout<<"PREORDER"<<endl;
cout<<"===== "<<endl;
PreOrder (Root) ;

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INORDER"<<endl;
cout<<"===== "<<endl;
InOrder (Root) ;

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"POSTORDER"<<endl;
cout<<"===== "<<endl;
PostOrder (Root) ;

cout<<endl;
cout<<endl;

cout<<"===== "<<endl;
cout<<"INFORMASI TREE"<<endl;
cout<<"===== "<<endl;
cout<<"Jumlah Node   : "<<HitungNode (Root)<<endl;
cout<<"Jumlah Daun   : "<<HitungDaun (Root)<<endl;
cout<<"Tinggi Tree    : "<<TinggiTree (Root)<<endl;

cout<<endl;
system("pause");
return 0;

```

1 }

- SS Running Program

```
D:\ASD SMT2\ASD-A-25-26\PRAK10_BTtree_Asprak01_4525210035_Khairul Adam Efendi>PASD10-11
=====
          PROGRAM BINARY TREE
=====
Masukkan Root : 7
Root Berhasil Dibuat

Masukkan Anak Kiri dari 7 (0 jika tidak ada) : 1
Masukkan Anak Kanan dari 7 (0 jika tidak ada) : 2
Masukkan Anak Kiri dari 1 (0 jika tidak ada) : 3
Masukkan Anak Kanan dari 1 (0 jika tidak ada) : 4
Masukkan Anak Kiri dari 2 (0 jika tidak ada) : 0
Masukkan Anak Kanan dari 2 (0 jika tidak ada) : 0
Masukkan Anak Kiri dari 3 (0 jika tidak ada) : 0
Masukkan Anak Kanan dari 3 (0 jika tidak ada) : 0
Masukkan Anak Kiri dari 4 (0 jika tidak ada) : 0
Masukkan Anak Kanan dari 4 (0 jika tidak ada) : 0

=====
ISI BINARY TREE (LEVEL ORDER)
=====
7 1 2 3 4

=====
PREORDER
=====
7 1 3 4 2

=====
INORDER
=====
3 1 4 7 2

=====
POSTORDER
=====
3 4 1 2 7

=====
INFORMASI TREE
=====
Jumlah Node   : 5
Jumlah Daun   : 3
Tinggi Tree   : 3

Press any key to continue . . .
```